APPENDIX E

Further Reading

Every link below was opened and checked on 2026-10-12. Documentation moves, and pages about unstable features change quickly; if a link is broken, search for the title.

E.1 Official documentation

The Rust Programming Language ("the Book"), chapter "Validating References with Lifetimes": the introduction most readers start from. https://doc.rust-lang.org/book/ch10-03-lifetime-syntax.html

The Rust Reference describes the language precisely. The pages most relevant to this book:

- Lifetime elision, including default trait-object lifetimes (Chapters 3, 6 and 8): https://doc.rust-lang.org/reference/lifetime-elision.html — the section on trait-object defaults is known to be inaccurate about precedence (see https://github.com/rust-lang/reference/issues/1407, the open pull request https://github.com/rust-lang/reference/pull/2281 that rewrites it, and Chapter 8).
- Subtyping and variance (Chapter 4): https://doc.rust-lang.org/reference/subtyping.html
- Trait and lifetime bounds, including higher-ranked trait bounds (Chapter 9): https://doc.rust-lang.org/reference/trait-bounds.html#higher-ranked-trait-bounds
- `impl Trait`, including precise capturing with `use<..>` (Chapter 8): https://doc.rust-lang.org/reference/types/impl-trait.html

The Rustonomicon is the guide to `unsafe` Rust, and its chapters on lifetimes are the classic deeper treatment:

- Lifetimes: https://doc.rust-lang.org/nomicon/lifetimes.html
- Subtyping and variance: https://doc.rust-lang.org/nomicon/subtyping.html
- Higher-rank trait bounds: https://doc.rust-lang.org/nomicon/hrtb.html
- Unbounded lifetimes (Chapter 13): https://doc.rust-lang.org/nomicon/unbounded-lifetimes.html

The Edition Guide explains what each edition changed. For edition 2024:

- RPIT lifetime capture rules (Chapter 8): https://doc.rust-lang.org/edition-guide/rust-2024/rpit-lifetime-capture.html
- Tail expression temporary scope (Chapter 5): https://doc.rust-lang.org/edition-guide/rust-2024/temporary-tail-expr-scope.html
- `if let` temporary scope (Chapters 5 and 12): https://doc.rust-lang.org/edition-guide/rust-2024/temporary-if-let-scope.html

The rustc book lists every lint, including `mismatched_lifetime_syntaxes` (Chapter 6): https://doc.rust-lang.org/rustc/lints/listing/warn-by-default.html#mismatched-lifetime-syntaxes and the lint groups such as `rust-2018-idioms`: https://doc.rust-lang.org/rustc/lints/groups.html

The release notes list every change, including the small lifetime fixes mentioned in Chapter 3 (associated consts in 1.99): https://doc.rust-lang.org/stable/releases.html

The error index has an entry for every error code; `rustc --explain E0597` prints the same text locally. For example, E0794 (Chapter 9): https://doc.rust-lang.org/error_codes/E0794.html

E.2 The compiler

The rustc development guide documents how the compiler works. Two of its pages are references for this book:

- Two-phase borrows (Chapter 5): https://rustc-dev-guide.rust-lang.org/borrow-check/two-phase-borrows.html
- Early-bound and late-bound parameters (Chapter 9): https://rustc-dev-guide.rust-lang.org/early-late-parameters.html

The pull request that changed default trait-object lifetimes in Rust 1.98 (Chapter 8): https://github.com/rust-lang/rust/pull/129543

The pull request that refined elision for receivers that contain `Self` (Chapter 3, Rust 1.81): https://github.com/rust-lang/rust/pull/117967

E.3 RFCs

The design documents behind the features in this book:

- RFC 1066, making `mem::forget` safe, the reason `thread::scope` takes a closure (Chapter 13): https://rust-lang.github.io/rfcs/1066-safe-mem-forget.html
- RFC 2094, non-lexical lifetimes, including "Problem case #3" (Chapters 1 and 5): https://rust-lang.github.io/rfcs/2094-nll.html
- RFC 3216, closure lifetime binders (Chapter 9, unstable): https://rust-lang.github.io/rfcs/3216-closure-lifetime-binder.html
- RFC 3498, lifetime capture rules for edition 2024 (Chapter 8): https://rust-lang.github.io/rfcs/3498-lifetime-capture-rules-2024.html
- RFC 3617, precise capturing with `use<..>` (Chapter 8): https://rust-lang.github.io/rfcs/3617-precise-capturing.html

E.4 The Rust blog

- Announcing Rust 1.31 and Rust 2018, the release that brought non-lexical lifetimes (Chapter 1): https://blog.rust-lang.org/2018/12/06/Rust-1.31-and-rust-2018/

- Generic associated types to be stable in Rust 1.65 (Chapter 10): https://blog.rust-lang.org/2022/10/28/gats-stabilization/
- Announcing Rust 1.85.0 and Rust 2024, with async closures and the `AsyncFn` traits (Chapters 9 and 11): https://blog.rust-lang.org/2025/02/20/Rust-1.85.0/
- The longer explanation of async closures and why `F: Fn(&T) -> Fut` cannot express borrowing callbacks: https://blog.rust-lang.org/inside-rust/2024/08/09/async-closures-call-for-testing/
- Enabling Polonius Alpha on nightly (Chapter 5): https://blog.rust-lang.org/2026/08/04/enabling-polonius-alpha-on-nightly/
- Enabling the next-generation trait solver on nightly: https://blog.rust-lang.org/2026/08/21/enabling-next-solver-on-nightly/

E.5 Unstable features mentioned in the book

Tracking issues are the place to check the current status:

- `closure_lifetime_binder`: https://github.com/rust-lang/rust/issues/97362
- `non_lifetime_binders`: https://github.com/rust-lang/rust/issues/108185
- `return_type_notation`: https://github.com/rust-lang/rust/issues/109417
- Polonius: https://github.com/rust-lang/rust/issues/160456
- `dropck_eyepatch` (`#[may_dangle]`): https://github.com/rust-lang/rust/issues/34761
- Defaults for `where Self: 'a` on GATs: https://github.com/rust-lang/rust/issues/87479
- The 2026 project goal on region constraints (bounds on higher-ranked lifetimes, implied bounds): https://goals.rust-lang.org/2026/next-gen-region-constraints.html

E.6 Beyond the official documentation

- serde, "Understanding deserializer lifetimes": the `Deserialize<'de>` / `DeserializeOwned` design of Chapter 9 explained by its authors. https://serde.rs/lifetimes.html
- The documentation of tokio's `spawn`, a typical executor signature (`F: Future + Send + 'static`; Chapter 11): https://docs.rs/tokio/latest/tokio/task/fn.spawn.html
- Miri, the undefined-behavior detector for `unsafe` code (Chapter 13): https://github.com/rust-lang/miri
- Sabrina Jewson, "The Better Alternative to Lifetime GATs": the helper-trait pattern of Chapter 10. https://sabrinajewson.org/blog/the-better-alternative-to-lifetime-gats